

SME review — the one hour we need from Vishnu

Owner: Harshan · **Reviewer:** Vishnu · **Status:** not yet held **Purpose:** close the questions that only a refrigeration engineer can close, in the order of what a wrong answer costs.

Read this first

This is a **merged** agenda. Two question sets existed: 36 in this repository, and 57 in the Thermynx FDD initiative at `docs/plan-v4.9.1/fdd/04-sme/questions.md`. They overlapped. Asking the same threshold twice would spend an hour that cannot be bought back, so the duplicates are resolved here and the reconciliation is recorded in §6.

The framing that works — and it is theirs, not ours: *break these, not review these*. We are most confident about §1.1, which is exactly why being wrong there costs the most.

"I do not know" and "it depends" are useful answers. If a test is not reliable we would far rather show two possible causes and let the technician decide than eliminate the right one and sound certain about it.

Why this is the last gate. The platform does not just list checks. It uses the answers to **rule causes out**. If one of our tests is wrong, the system eliminates the *correct* cause — confidently — and nobody re-examines a settled question. Elimination is irreversible in this flow.

§1 · Irreversible if wrong — spend the hour here

RC2 F5 F7 — these decide whether the platform may eliminate a candidate cause on judgement nobody qualified has checked.

1.1 Is the filter-drier temperature drop really the deciding test?

When suction pressure is far below expected but current is barely up, we read it as a **starved evaporator** rather than a compressor working too hard. To tell *why* it is starved we ask for the temperature drop across the filter-drier. The reasoning: a blockage makes refrigerant expand and go cold, so a cold spot means a restriction and no cold spot means the charge is low.

- Can an **undercharged** circuit also produce a cold spot across the drier?
- Is it **safe to measure while the machine is running**?

If the answer is no: the test does not separate the two causes, and we are eliminating the wrong one. **F7** keeps them as a combined label and the differential loses its first question.

This is the one we are most confident about, it is the first question asked on six fault episodes, and it is the example in the demo.

Answer:

1.2 Can condenser leaving-water temperature separate the two fault families?

Measured averages:

Fault type	Condenser leaving water
Condenser low flow	34.9 °C
Condenser water side	34.5 °C
High head, cause unclear	32.4 °C
Power high, unexplained	31.0 °C
Refrigerant side high head	30.1 °C

About a **4 °C gap** between the water-side faults and the refrigerant-side one.

- Is that a **real physical difference** we can rely on, or a coincidence of when those faults happened to occur?

If it is real: whole cause groups can be ruled out from data already held, with nobody sent to measure anything. **If not:** it stays a *prior* — it may reorder the questions and nothing else.

Answer:

1.3 Does an unusually equal split between two compressors mean a compressor fault?

Each chiller has two compressors metered separately. Normally the second draws about **1.17×** the first.

Fault type	Ratio
Compressor inefficiency	1.03 — unusually equal
Everything else	1.09 – 1.20

- Does *unusually equal* point at a compressor fault, or is it normal staging, lead/lag rotation or unloading?

If it is real: it replaces a three-day oil analysis with a reading already held.

Answer:

1.4 Are the measured separations strong enough to eliminate on?

F7 currently keeps two pairs combined — undercharge with restriction, overcharge with non-condensables — because *automatic elimination on unreviewed thresholds is more dangerous than the ambiguity it removes*. The separations in 1.2 and 1.3 were measured and deliberately withheld.

- Do they become **eliminators**, or stay **priors** that only reorder questions?

Answer:

1.4a How the narrowing works — so you can judge one line at a time

Four differentials, 19 candidate causes, 19 discriminating questions. Each answer does one of three things:

Effect	Meaning	Reversible
confirm	evidence for this cause	—
eliminate	rules it out	no
keep	consistent, decides nothing	—

Three properties are already built in, and we would like you to check the reasoning rather than the code:

- **A confirmation never eliminates its siblings.** A fouled condenser on a machine that is also low on flow is two real causes, and collapsing to the first confirmation is how the second gets missed.
- **"Can't tell" does nothing.** Every question carries it explicitly, with no effect, because otherwise uncertainty would quietly eliminate something.
- **Every elimination records the check and the answer that caused it** — so *"why did nobody look at the tower?"* has a better answer than *"the software decided"*.

Thirty-one causes have already been ruled out on the live queue, every one by a discriminator nobody qualified has reviewed. That is the whole reason for this hour.

Answer / corrections, one line at a time:

1.4b The three lines that carry most of the risk

If the hour runs short, answer these three and stop. Roughly forty-one eliminations exist across the four differentials; these three remove the most candidates per answer, and two of them sit on the questions most likely to be asked in production.

A • The commonest class, on its highest-power question

`HIGH_HEAD_AMBIGUOUS` is **430 of 674 fault slots — 64 %**, present on 12 of 12 fault days. Its first question is the condenser approach temperature against design, and answering **wider** eliminates the cooling tower, reasoned as *"the tower sets the water temperature, not the approach across it."*

- Can a tower fault raising entering-water temperature **coexist** with a widened approach? If it can, this is **keep**, not **eliminate** — and it is the elimination most likely to actually fire, because it is the lead question on two thirds of our faults.

Answer:

B · One answer that removes three of five causes

`CONDENSER_WATER_SIDE_UNSPECIFIED` asks for condenser water flow against design. Answering **at design** eliminates low flow, a blocked strainer *and* the pump in one go, reasoned as "*adequate flow with a failing condenser points at the surface, not the hydraulics.*"

Two things make this the sharpest line in the library. It removes 60 % of the differential on a single answer — and **the measurement does not come from telemetry**. Condenser flow has never recorded a non-zero value on the reference plant, so the checklist already says to *expect to measure it manually rather than read it*.

- Is the three-way elimination correct?
- Is a manual condenser-flow measurement realistic on your plants? **If it is not, what replaces it as the lead question?** This is the same instrumentation gap as `Q1` in §2 — here it decides a differential rather than a model.

Answer:

C · Four causes closed by one judgement, not a measurement

`POWER_HIGH_UNEXPLAINED` asks whether actual load is higher than the current curve explains. Answering **genuinely higher** eliminates **all four** electrical causes — phase imbalance, the motor, the drive and a loose connection — reasoned as "*if the load explains the current there is no electrical fault to chase, and it is the one question an operator can answer from the panel.*"

The reasoning holds only if an operator can reliably tell *the machine is working harder* from a panel. That is a judgement rather than a measurement, and it would be recorded as **estimated** — which is exactly the case §4.3 asks about in the general. Here it is concrete: one estimated answer closes the entire electrical branch, and the plant streams no phase currents, no voltages and no drive registers, so none of the four can be checked against data afterwards.

- Should this answer eliminate, or only **deprioritise** the electrical branch?
- If an estimate should not settle it, what measurement should?

Answer:

1.5 Are any causes missing from the candidate lists?

An absent cause can never be found, which is worse than a wrong elimination.

Answer:

1.6 Which faults are answered by stopping the machine, not by raising a work order?

The taxonomy we inherited has **no safety impact class** — every escalation route ends in a work order, so there was no way to say *stop now*. S6 exists to fix that and needs its trigger list.

Answer:

1.7 Is a drafted-but-unreviewed interim holding action worse than none?

When a fault is found and the repair must wait, the machine keeps running. Nine short instructions for running it safely in the meantime were drafted and **deliberately never switched on**, because telling somebody it is safe to keep running a machine is the most dangerous thing this system could say. The cost of that choice: a deferred critical fault runs with nothing protecting it.

- Would an AI-drafted holding action, **approved once per class by you**, be acceptable as library content?

Answer:

1.8 When five faults appear on one machine, how many problems is that?

This is the newest question and possibly the easiest one to answer. It comes from counting our own data rather than from a document, and the proposal we have written is **our inference, reviewed by nobody**.

What the data shows. In the measured window, twelve equipment-days carried a fault. Naming a case for each (*machine, day, fault label*) gives **thirty-nine**. And on **15 April, chiller 1 carried five labels at the same time**:

Label
CONDENSER_LOW_FLOW — the only class rated critical
HIGH_HEAD_AMBIGUOUS
POWER_HIGH_UNEXPLAINED
REFRIGERANT_SIDE_HIGH_HEAD
STARVED_EVAP_UNDERCHARGE_OR_RESTRICTION

17 April is another five, and **ten of chiller 1's twelve fault days carry more than one label**.

Why it matters more than it sounds. If each label opens its own case and each case raises its own work order, that is five work orders, five visits and five checklist runs. The technician fixes the real problem on visit one, so visits two to five find nothing and close as *no fault found* — which teaches the crew that the system cries wolf.

Our proposal, which we would like you to break: where several labels share a plausible common cause, group them into **one** investigation with **one** work order, and show the grouping to a human who can split it.

1.8a Can one cause really produce four of those at once? Our reasoning is that a **fouled condenser** raises head pressure, makes the compressor work harder, drops efficiency and shifts refrigerant-side pressures — so `HIGH_HEAD_AMBIGUOUS`, `POWER_HIGH_UNEXPLAINED`, `COMPRESSOR_INEFFICIENCY` and `REFRIGERANT_SIDE_HIGH_HEAD` would be four symptoms of one problem. **Is that right, and are the groups you would draw the same as ours?**

Answer / the groups you would draw:

1.8b Which pairs must never be grouped? The opposite mistake is the dangerous one. If a fouled condenser and a genuinely low refrigerant charge are both present and we group them, the second one is hidden — and a missed undercharge costs a compressor, where a duplicate visit costs a morning. **Which label pairs are independent enough that they must always stay separate cases, even when they appear together?**

Answer:

1.8c Two of the five contradict each other. What does that mean on the plant? `CONDENSER_LOW_FLOW` is one of only two classes whose signature has a **positive** discharge-pressure residual, which is how the water side gets implicated. `HIGH_HEAD_AMBIGUOUS` has a **negative** one, and that negative value is precisely the evidence used to argue the water side is *not* involved. Both were present on the same machine on the same day.

- Is that **two real faults** at different times of day, **one fault in transition**, or a sign the **data is not trustworthy** in that window?
- And which of the three should the platform assume when it cannot tell?

Answer:

1.8d If an instrument is faulty, can a real fault hide behind it? Our plan is that when an instrument fault appears in a group, the instrument case leads and the others wait for the sensor to be fixed. **Is that safe, or can a genuine fault sit underneath a bad sensor and be missed while we wait?**

Answer:

1.8e Is five labels on one machine-day normal? Or does it tell you the detector is too sensitive, and the real number of problems on 15 April was one?

Answer:

§2 · The data that does not exist — not your questions to answer, but yours to interpret

Four defects. Three of our checks ask for numbers that are not there. These need the site and instrumentation team, but only you can say what to do without them.

#	What we found	What it costs
2.1	cond_flow has never had a reading — not "died in May", zero non-zero values ever	This is the highest-leverage single measurement. It feeds four of the six models. Without it the entire efficiency and high-head branch returns NO_DIAGNOSIS
2.2	dpt never changes — a constant 107.0 on chiller 1, 112.9 on chiller 2	Condenser approach temperature cannot be computed at all. That is the fouling threshold <i>and</i> a question inside a differential. Is that tag a setpoint or design value rather than a measurement?
2.3	Condenser ΔT is negative every month on chiller 1 (−3.0 to −3.4)	A condenser rejects heat, so leaving water must be warmer than entering. Those two columns look swapped or mislabelled — and no residual on that machine can be trusted until it is resolved
2.4	Both chilled-water flow transmitters have read near zero since May while ΔT and power stayed normal	Physically impossible. It quietly invalidated two months of efficiency figures and blinded the fault model

2.5 If condenser flow cannot be measured, what leads instead? A manual measurement, a proxy, or do we accept **NO_DIAGNOSIS** on that branch and say so plainly?

Answer:

2.6 What you will and will not see in the demonstration, and why

Not a question — a disclosure, so that nothing in the demonstration surprises you.

Our copy of the plant data runs to 2026-08-05, but the **measured** data stops at **2026-06-23**. The six weeks after that are a simulation. Comparing the two windows column by column, 31 of 32 signals are a continuation of something the plant really measures. One is not: **cond_flow** has **zero** readings in 31,884 real slots and **3,354 fabricated ones** in the simulated window, reaching 893.7.

So the demonstration runs on the **measured** window, and the high-head branch will return **NO_DIAGNOSIS** where condenser flow is needed. **That is deliberate, and it is the honest result** — we would rather show you the platform refusing than show it confident on a measurement your plants cannot take.

If you would rather see the branch resolve, that needs an answer to 2.5 first, not a different window.

Anything you want shown differently:

§3 · Thresholds — the numbers we refuse to invent

Every one of these is unresolved in the documents, and carries no agreed number. We will not guess them.

#	Question	Our proposed starting point	If it is wrong
3.1	Minimum load for a valid diagnosis	confirm per machine type	gates pass on data the model cannot judge
3.2	Settling time after a start or stage change	—	residuals trusted during a transient
3.3	Settling time after a leaving-water setpoint change	—	same
3.4	Persistence window — one value, or per fault class?	20–30 minutes as a common default	transients become faults, or faults are missed
3.5	Healthy chilled-water ΔT band per machine	design ΔT from the machine data sheet	F14 fires wrongly or never
3.6	Condenser approach thresholds for fouling	per OEM data sheet	blocked — see 2.2, the input does not exist
3.7	Time constants separating low condenser flow from fouling	low flow spiky/intermittent; fouling steady drift over weeks	the two condenser-side causes cannot be told apart
3.8	Volatility threshold defining expansion-valve hunting	set from observed data	hunting missed or over-reported
3.9	Sensor drift thresholds per sensor type	per sensor class	a crew is sent for a drifting sensor
3.10	Is compressor lead/lag reliably available from the BMS?	—	five of six models shift at every stage change

Answers:

§4 · The checklist library — 124 items, one review pass

RC2 **RC3** — the library is **curated content, never model output**, because a checklist directs physical work on pressurised equipment and a plausible-but-wrong item is worse than no item. That is what makes this review possible at all.

What exists: 124 curated items across 11 fault classes, plus a 7-item generic fallback. Split **57 RCA · 37 corrective · 30 preventive**, of which **24 are blocking**. Four decision trees. Role tags: technician 49 · supervisor 38 · operator 29 · maintenance 7 · vendor 1.

One review pass has ever been run, over one class. It found an oil analysis — acid number, moisture, metals — being shown to whoever opened a compressor case. That is a lab task, and it produced the rule that governs the whole role model: *an operator must never be blocked by a check they cannot perform.*

4.1 Is the maintenance/technician split drawn in the right place for this crew?

Only **7 of 124** items are tagged `maintenance`, yet brushing condenser tubes, cleaning strainers, venting a loop and servicing a tower are in-house mechanical work and several of those are tagged `technician`. Untagged items default to `technician` deliberately — over-escalating wastes a callout, under-escalating puts an unqualified person on a pressurised circuit.

- Are any obviously in the wrong bucket?

Answer:

4.2 How does a technician say "I found nothing" versus "I found something"?

When every check comes back negative the platform must stop and **not** produce a root cause, because a conclusion built on "nothing wrong anywhere" is misleading. Detecting that from wording is crude. It currently reads this as a **negative**:

"Measured 6 °C drop across the drier — clear cold spot"

...because of the word *clear*. That is a strong positive finding and we would be throwing it away.

- What words do technicians actually use for **"nothing here"**?
- Which words look negative but are really **positive findings**?

Answer:

4.3 Does an estimate settle a blocking check, or only a measurement?

Our position is **measured only**. An untagged answer once defaulted to *estimated* and opened a blocking gate; separately, six "N/A" presses opened one with no evidence behind it. `RC10` records three answers rather than two — measured, estimated, and cannot-check — and only the first satisfies a blocking item.

Answer:

4.3a Does `REFRIGERANT_SIDE_HIGH_HEAD` need a differential?

It names a *region* rather than a mechanism, probes five mechanisms, and has **no blocking items and no differential** — so a case can conclude there having gathered no evidence at all. Should it get one?

Answer:

4.4 Should every fault class have a verification step?

Three of seven equipment classes have none. **This question has no counterpart in the Thermynx set, because that implementation has no verification layer at all** — and proving the repair worked is the promise this MVP is built on.

- What proves a **condenser cleaning** worked? Our proposal: `rPwr`, `rDP` and `rCWL` back inside band, held several days across a representative load range.
- What verifies **compressor inefficiency** and **power high unexplained**?

Answer:

4.5 Who approves a recurring obligation?

Thirty preventive items exist and none of them creates a schedule — prevention is recorded and never happens. `RC11` makes a preventive item a real commitment, which means somebody signs for future work.

Answer:

4.6 Is there a plant document to reconcile against?

An OEM manual or O&M schedule the 124 items could be **cross-checked against** rather than reviewed from scratch. This could turn an hour of review into a much shorter cross-check.

Answer:

§5 · Not for Vishnu — the decision Harshan owes

#	Question	Blocks
5.1	Is the MVP cut agreed? 94 of 147 features	<code>mvp/BACKLOG.md</code> and all build sequencing
5.2	How do Synex and Thermynx relate?	every line of positioning copy
5.3	Which of the three inherited gaps do we take on — no holding action, no retraction mechanism, duplicated checklist work under event grouping?	<code>F9</code> , <code>v7</code> , deferred-fault risk
5.4	Four MVP features are named by no build stage, two of them safety	the stage-9 estimate
5.5	Three role systems — user personas, capability roles, agent skills. Which governs which decision?	<code>C11</code> , <code>C20</code> , <code>RC3</code>

§6 · Reconciliation — what was merged, and what is uniquely ours

The two sets overlapped substantially. Resolved as follows:

Ours	Theirs	Resolution
Q1 condenser flow trustworthy	Q-K4, data defect	merged → §2.1, §2.5
Q2 evaporator flow derivation	Q-A11	merged → §2.4
Q8 condenser approach thresholds	data defect dpt	merged → 3.6, blocked on 2.2
Q10, Q11 keep the pairs combined	Q-K1, Q-K2	merged → §1.1, §1.4
Q20 missing fault classes	Q-K6	merged → §1.5
Q22 stop-the-machine faults	Q-T3	merged → §1.6
Q23 estimated settles a gate	Q-C4	merged → §4.3
Q33 words for "nothing found"	their question 4	merged → §4.2
Q34 maintenance/technician split	Q-N1	merged → §4.1
Q35 who approves a PM obligation	Q-N2	merged → §4.5
Q36 reconcile against a plant document	Q-N5	merged → §4.6
—	Q-B4 interim holding actions	adopted → §1.7
Q15 what proves a repair worked	<i>no counterpart</i>	uniquely ours → §4.4

Theirs that stay theirs: the platform and data questions in their §2 — residual flag semantics, model refit ownership, whether the classifier is input-gated, the corrupted subsystem join. Those need their platform team, not this review. Ours Q24 – Q28 are the equivalents on our side.

The one that matters most for this programme is §4.4. Everything else sharpens a diagnosis; that one is the difference between a platform that says a repair worked and one that proves it.

How to answer

Whatever is easiest — a reply in your own words, one line per question, or writing under the **> Answer:** lines above, or a call where we write it down for you.

Every mapping you correct is **one line** in the library. It was built that way deliberately, so a correction never touches the machinery.

When answers arrive

1. Record each in `decisions/DECISIONS.md` in **his words**, with the alternative that was rejected — a decision without its rejected alternative cannot be revisited.
2. Move the closed rows in `decisions/OPEN-QUESTIONS.md` to the Closed section. Do not delete them.

3. Update the affected register rows and `CONTEXT.md`.
4. Re-run `python scripts/verify.py` and `python scripts/sync_mvp_html.py`.
5. Append a row to `HANDOFF.md` under Recent changes.